Part 1: Theoretical Questions

Question 1.1

1. `{f:(T1 -> T2), g:(T1 -> T2), a: T1} ⊢ (f (g a)):T2`

False. `g : (T1 -> T2)` and `a : T1`, so `(g a) : T2`. However, `f : (T1 -> T2)` expects an argument of type `T1`, not `T2`. Since `T1` and `T2` are not necessarily the same type, the application `(f (g a))` is not well-typed.

2. `{f:(T1 * T2 -> T3)} ⊢ (lambda (x) (f x y)): (T2 -> T3)`

False. `y` is a free variable that does not appear in the type environment, so the expression is not well-typed. Additionally, `(f x y)` applies `f` to two arguments where `x` would need type `T1`, but the lambda parameter `x` is typed as `T2` in the return type — the parameter types are inconsistent.

3. `{f:(T1 * T2 -> T3), y: T2} ⊢ (lambda (x) (f x y)):(T1 -> T3)`

True. `y : T2` is in the environment. The lambda binds `x`, and in the body `(f x y)`, `x : T1` and `y : T2`, which matches `f : (T1 * T2 -> T3)`. So `(f x y) : T3`, and the lambda has type `(T1 -> T3)`.

Question 1.2

1.2.1: `(lambda (f1 g1 x1 y1) (f1 (g1 y1 x1) (g1 x1 y1)))`

(i) Pool:

TVar	Sub-expression
T0	`(lambda (f1 g1 x1 y1) (f1 (g1 y1 x1) (g1 x1 y1)))`
T1	`(f1 (g1 y1 x1) (g1 x1 y1))`
Tf1	`f1`
Tg1	`g1`

TVar	Sub-expression
Tx1	`x1`
Ty1	`y1`
T2	`(g1 y1 x1)`
T3	`(g1 x1 y1)`

(ii) Equations:

I Lambda: `T0 = [Tf1 * Tg1 * Tx1 * Ty1 -> T1]`

II Application `(f1 (g1 y1 x1) (g1 x1 y1))`: `Tf1 = [T2 * T3 -> T1]`

III Application `(g1 y1 x1)`: `Tg1 = [Ty1 * Tx1 -> T2]`

IV Application `(g1 x1 y1)`: `Tg1 = [Tx1 * Ty1 -> T3]`

(iii) The inference succeeds.

From equations (3) and (4): `[Ty1 * Tx1 -> T2] = [Tx1 * Ty1 -> T3]`, which gives:

- `Ty1 = Tx1`
- `T2 = T3`

Final substitution:

- `Ty1 = Tx1`
- `T2 = T3`
- `Tg1 = [Tx1 * Tx1 -> T2]`
- `Tf1 = [T2 * T2 -> T1]`
- `T0 = [[T2 * T2 -> T1] * [Tx1 * Tx1 -> T2] * Tx1 * Tx1 -> T1]`

1.2.2: `((lambda (f1) (lambda (x1) (f1 x1 x1))) (lambda (y1) y1))`

(i) Pool:

TVar	Sub-expression
T0	`((lambda (f1) (lambda (x1) (f1 x1 x1))) (lambda (y1) y1))`
T1	`(lambda (f1) (lambda (x1) (f1 x1 x1)))`
T2	`(lambda (x1) (f1 x1 x1))`

TVar	Sub-expression
T3	`(lambda (y1) y1)`
T4	`(f1 x1 x1)`
Tf1	`f1`
Tx1	`x1`
Ty1	`y1`

(ii) Equations:

I Lambda: `T1 = [Tf1 -> T2]`

II Lambda: `T2 = [Tx1 -> T4]`

III Lambda: `T3 = [Ty1 -> Ty1]`

IV Application `((lambda (f1) (lambda (x1) (f1 x1 x1))) (lambda (y1) y1))`: `T1 = [T3 -> T0]`

V Application `(f1 x1 x1)`: `Tf1 = [Tx1 * Tx1 -> T4]`

(iii) The inference fails.

The operand `(lambda (y1) y1)` is the identity function, so from equation (3) it has the one-parameter type `T3 = [Ty1 -> Ty1]`.

Combining equations (1) and (4): `[Tf1 -> T2] = [T3 -> T0]`, which gives `Tf1 = T3 = [Ty1 -> Ty1]`. So `f1` is a procedure with **one** parameter.

But in the body `(f1 x1 x1)`, `f1` is applied to **two** arguments, so equation (5) forces `Tf1 = [Tx1 * Tx1 -> T4]`, a procedure with **two** parameters.

Unification must then solve the conflicting equation:

```
[Ty1 -> Ty1] = [Tx1 * Tx1 -> T4]
```

These are two procedure types of different arity (1 vs 2). `canUnify` requires equal numbers of parameters, so it cannot unify them and the algorithm reports "*Equation contains incompatible types*". The inference fails because `f1` is bound to the identity (one parameter) yet applied to two arguments.

1.2.3: `((lambda (f1) (lambda (x1) ((f1 x1) x1))) (lambda (y1) y1))`

(i) Pool:

TVar	Sub-expression
T0	`((lambda (f1) (lambda (x1) ((f1 x1) x1))) (lambda (y1) y1))`
T1	`(lambda (f1) (lambda (x1) ((f1 x1) x1)))`
T2	`(lambda (x1) ((f1 x1) x1))`
T3	`(lambda (y1) y1)`
T4	`((f1 x1) x1)`
T5	`(f1 x1)`
Tf1	`f1`
Tx1	`x1`
Ty1	`y1`

(ii) Equations:

I Lambda: `T1 = [Tf1 -> T2]`

II Lambda: `T2 = [Tx1 -> T4]`

III Lambda: `T3 = [Ty1 -> Ty1]`

IV Application `((lambda (f1) (lambda (x1) ((f1 x1) x1))) (lambda (y1) y1))`: `T1 = [T3 -> T0]`

V Application `(f1 x1)`: `Tf1 = [Tx1 -> T5]`

VI Application `((f1 x1) x1)`: `T5 = [Tx1 -> T4]`

(iii) The inference fails (occur-check / circular type).

Unlike 1.2.2, the body here is the **curried** application `((f1 x1) x1)`: `f1` is applied to a single argument `x1`, and the result is then applied to `x1`. So `f1` is used as a one-parameter procedure, consistent with the identity operand — there is **no arity conflict** this time.

Tracing the unification:

- The operand `(lambda (y1) y1)` is the identity, so by equation (3), `T3 = [Ty1 -> Ty1]`.
- Combining (1) and (4): `[Tf1 -> T2] = [T3 -> T0]`, giving `Tf1 = T3 = [Ty1 -> Ty1]`.

- Combining with (5): `[Ty1 -> Ty1] = [Tx1 -> T5]`, giving `Ty1 = Tx1` and `T5 = Ty1`. Hence `T5 = Tx1`.
- Substituting `T5 = Tx1` into equation (6) `T5 = [Tx1 -> T4]` yields the conflicting equation:

```
Tx1 = [Tx1 -> T4]
```

The algorithm now tries to bind the type variable `Tx1` to a procedure type that **contains** `Tx1` **itself**. The occur-check in `checkNoOccurrence` detects this circular reference and the algorithm reports:

```
Occur check error - circular sub Tx1 in (Tx1 -> T4)
```

Intuitively, since `f1` is the identity, `(f1 x1)` evaluates to `x1`, so `((f1 x1) x1)` is the self-application `(x1 x1)`. This forces `x1` to be a function whose own domain equals its own type — the infinite type `Tx1 = [Tx1 -> ...]`, which has no finite solution. The inference therefore fails.

1.3: `(lambda (f) (f (right (lambda (s) (f (left s))))))`

(i) Pool:

TVar	Sub-expression
T0	`(lambda (f) (f (right (lambda (s) (f (left s))))))`
T1	`(f (right (lambda (s) (f (left s)))))`
T2	`(right (lambda (s) (f (left s))))`
T3	`(lambda (s) (f (left s)))`
T4	`(f (left s))`
T5	`(left s)`
Tf	`f`
Ts	`s`

(`Tx` and `Ty` below are the fresh type variables introduced by the `right` and `left` rules.)

(ii) Equations:

I Lambda: `T0 = [Tf -> T1]`

II Lambda: `T3 = [Ts -> T4]`

III Application `(f (right (lambda (s) (f (left s)))))`: `Tf = [T2 -> T1]`

IV Right form `(right (lambda (s) (f (left s))))`: `T2 = Tx + T3`

V Application `(f (left s))`: `Tf = [T5 -> T4]`

VI Left form `(left s)`: `T5 = Ts + Ty`

(iii) The inference succeeds.

`f` is applied to two operands — `(right (lambda (s) (f (left s))))` and `(left s)` — so from equations (3) and (5) its single parameter type must be the same:

```
[T2 -> T1] = [T5 -> T4]  =>  T2 = T5,  T1 = T4
```

Unifying the two sum types bound to that parameter (equations 4 and 6):

```
Tx + T3 = Ts + Ty  =>  Tx = Ts,  Ty = T3 = [Ts -> T4]
```

The `left` injection fixes the left summand to `Ts`, and the `right` injection fixes the right summand to `[Ts -> T4]`. The sum type `(Ts + [Ts -> T4])` accommodates both injections, so there is no conflict — and no type variable occurs within its own binding, so there is no occur-check failure.

Final substitution (free variables `Ts`, `T4`):

Var	Resolves to
T1	`T4`
Tx	`Ts`
T3, Ty	`[Ts -> T4]`
T2, T5	`(Ts + [Ts -> T4])`
Tf	`[(Ts + [Ts -> T4]) -> T4]`
T0	`[[(Ts + [Ts -> T4]) -> T4] -> T4]`

So the whole expression has type `[[(Ts + [Ts -> T4]) -> T4] -> T4]`.